## `k8s-05` tasks

1. **Create a namespace dev-environment and apply a resource-based quota that restricts the number of pods to 3 and services to 2.**

# 1. Objective

To create a Kubernetes namespace named `dev-environment` and apply a ResourceQuota to restrict the maximum number of Pods and Services.

Requirements:

- Maximum Pods: 3
- Maximum Services: 2

# 2. Step-by-step implementation

### Step 1: Check existing namespaces

Command:

```
None
kubectl get namespaces
```

This command displays all namespaces available in the Kubernetes cluster.

### Step 2: Create the namespace

Command:

```
None
kubectl create namespace dev-environment
```

Output:

```
None
namespace/dev-environment created
```

Verification:

```
None
kubectl get namespaces
```

Result: `dev-environment` was created successfully.

### Step 3: Create ResourceQuota YAML file

Create a YAML file:

```
None
nano dev-quota.yaml
```

Add the following configuration:

```
None
apiVersion: v1
kind: ResourceQuota
metadata:
  name: dev-resource-quota
  namespace: dev-environment
spec:
  hard:
    pods: "3"
    services: "2"
```

Explanation:

| YAML field | Description |
| --- | --- |
| `apiVersion: v1` | Uses Kubernetes core API |

| | |
|---|---|
| `kind: ResourceQuota` | Creates a ResourceQuota object |
| `name` | ResourceQuota name |
| `namespace` | Namespace where quota is applied |
| `pods: "3"` | Allows maximum 3 Pods |
| `services: "2"` | Allows maximum 2 Services |

### Step 4: Apply ResourceQuota

Command:

```
None
kubectl apply -f dev-quota.yaml
```

Output:

```
None
resourcequota/dev-resource-quota created
```

### Step 5: Verify ResourceQuota

Command:

```
None
kubectl describe resourcequota dev-resource-quota -n
dev-environment
```

Initial output:

```
None
      Name:       dev-resource-quota
      Namespace:  dev-environment

      Resource    Used    Hard
      --------    ----    ----
      pods        0       3
      services    0       2
```

This confirms that the quota was applied successfully.

## 3. Test Pod quota restriction

Create three Pods:

```
None
      kubectl run test-pod-1 --image=nginx -n dev-environment
      kubectl run test-pod-2 --image=nginx -n dev-environment
      kubectl run test-pod-3 --image=nginx -n dev-environment
```

All three Pods were created successfully.

Now try creating a fourth Pod:

```
None
      kubectl run test-pod-4 --image=nginx -n dev-environment
```

Output:

```
None
      Error from server (Forbidden):
      exceeded quota: dev-resource-quota,
```

```
requested: pods=1,
used: pods=3,
limited: pods=3
```

Explanation: Kubernetes rejected the fourth Pod because the namespace already reached its maximum Pod limit of three.

Verification:

```
None
kubectl get pods -n dev-environment
```

All three Pods showed `Running`.

## 4. Test Service quota restriction

Create two Services:

```
None
kubectl create service clusterip test-service-1 --tcp=80:80 -n
dev-environment


kubectl create service clusterip test-service-2 --tcp=80:80 -n
dev-environment
```

Both Services were created successfully.

Now try creating a third Service:

```
None
kubectl create service clusterip test-service-3 --tcp=80:80 -n
dev-environment
```

Output:

```
None
      error: failed to create ClusterIP service:
      services "test-service-3" is forbidden:
      exceeded quota: dev-resource-quota,
      requested: services=1,
      used: services=2,
      limited: services=2
```

Explanation: Kubernetes rejected the third Service because the maximum allowed Service count is two.

Verification:

```
None
      kubectl get services -n dev-environment
```

Both Services were created successfully.

## 5. Clean up temporary testing resources

After completing the tests, delete all temporary Pods and Services.

Commands:

```
None
      kubectl delete pods --all -n dev-environment


      kubectl delete services --all -n dev-environment
```

Output confirmed that all three Pods and both Services were deleted.

Final verification:

```
None
      kubectl get all -n dev-environment
```

```
kubectl describe resourcequota dev-resource-quota -n
dev-environment
```

Final output:

```
None
No resources found in dev-environment namespace.

Name:       dev-resource-quota
Namespace:  dev-environment

Resource    Used    Hard
--------    ----    ----
pods        0       3
services    0       2
```

## 6. Final result

Task successfully completed

| Requirement | Expected | Actual result |
|---|---|---|
| Namespace | dev-environment | Created |
| Maximum Pods | 3 | Verified |
| Maximum Services | 2 | Verified |
| Fourth Pod creation | Rejected | Passed |

| | | |
|---|---|---|
| Third Service creation | Rejected | Passed |
| Cleanup | Temporary resources deleted | Completed |
| ResourceQuota | Remains configured | Verified |

## 7. Important concepts learned

1. Namespace: Logically separates resources inside a Kubernetes cluster.
2. ResourceQuota: Restricts resource consumption within a namespace.
3. Object-count quota: Limits the number of Kubernetes objects such as Pods and Services.
4. Hard limit: Maximum allowed quantity.
5. Used: Current quantity consumed.
6. Forbidden error: Kubernetes rejects resource creation when the quota is exceeded.

# 8. Interview questions

Q1. What is ResourceQuota?

ResourceQuota limits the total resource consumption or object count inside a Kubernetes namespace.

Q2. Can we apply ResourceQuota to a complete cluster?

No. ResourceQuota is applied at the namespace level.

Q3. What happens when a Pod quota is exceeded?

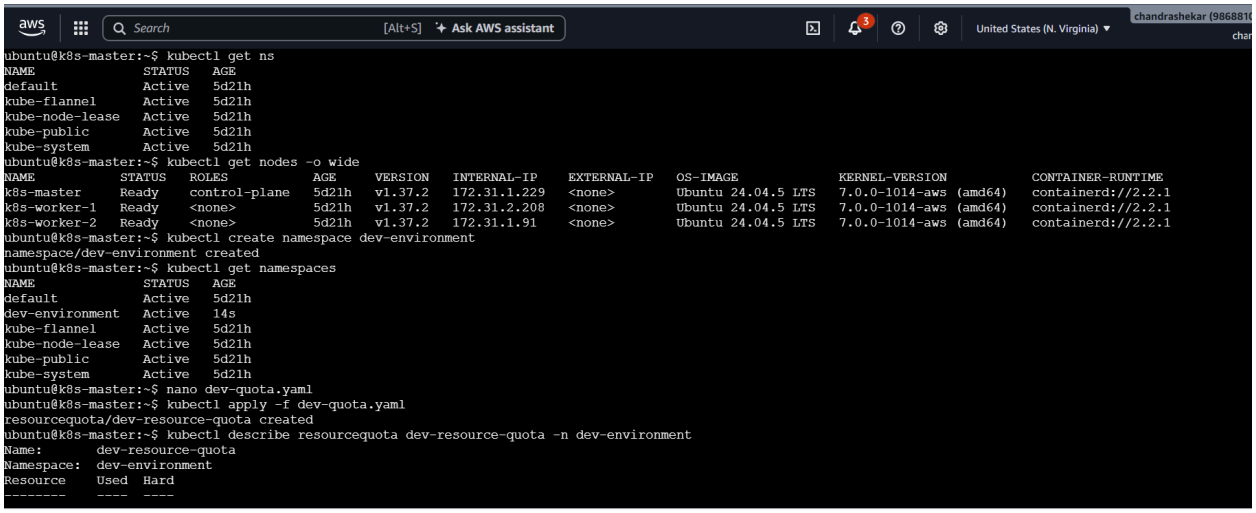
Kubernetes rejects the new Pod and returns a Forbidden quota error.

Q4. What is the difference between Used and Hard?

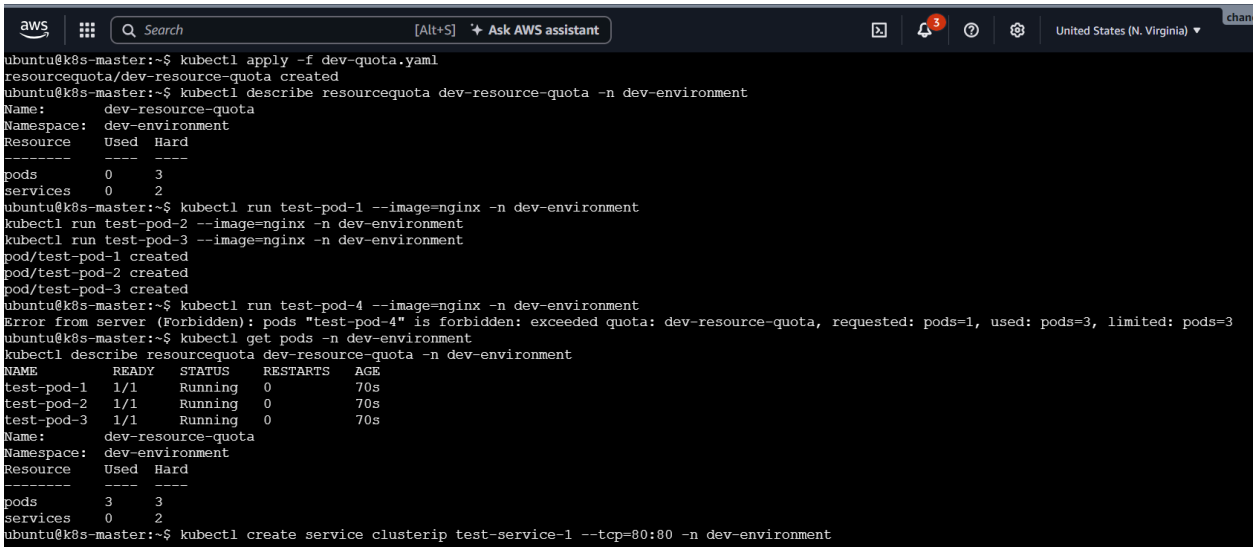
- Used: Current consumption.
- Hard: Maximum allowed consumption.

Q5. Does deleting Pods remove ResourceQuota?

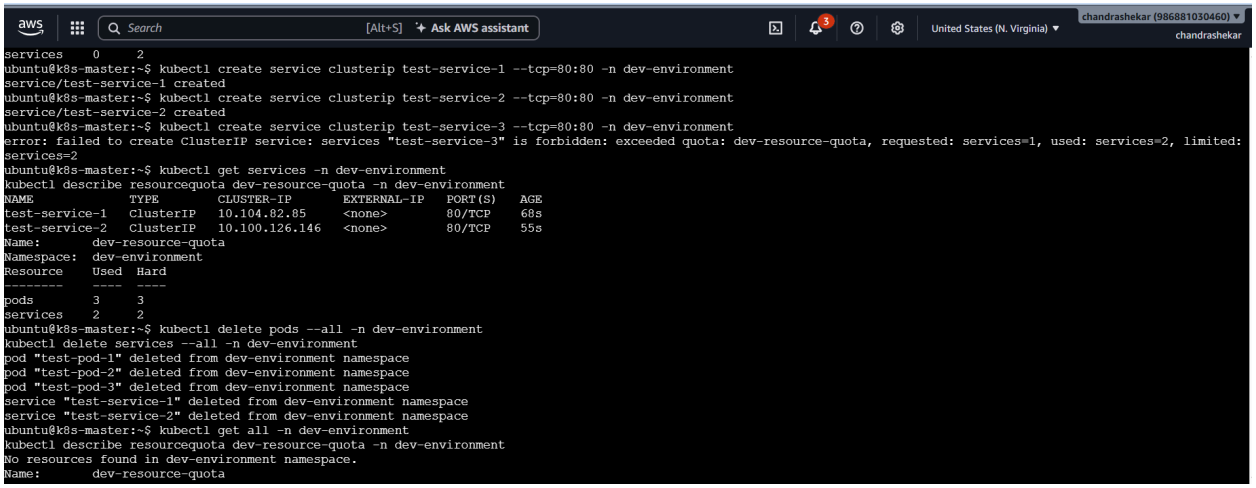
No. Deleting Pods does not delete ResourceQuota. The quota remains until explicitly deleted.



```
ubuntu@k8s-master:~$ kubectl get ns
NAME              STATUS   AGE
default           Active   5d21h
kube-flannel      Active   5d21h
kube-node-lease   Active   5d21h
kube-public       Active   5d21h
kube-system       Active   5d21h
ubuntu@k8s-master:~$ kubectl get nodes -o wide
NAME           STATUS   ROLES           AGE     VERSION   INTERNAL-IP    EXTERNAL-IP   OS-IMAGE             KERNEL-VERSION           CONTAINER-RUNTIME
k8s-master     Ready    control-plane   5d21h   v1.37.2   172.31.1.229   <none>        Ubuntu 24.04.5 LTS   7.0.0-1014-aws (amd64)   containerd://2.2.1
k8s-worker-1   Ready    <none>          5d21h   v1.37.2   172.31.2.208   <none>        Ubuntu 24.04.5 LTS   7.0.0-1014-aws (amd64)   containerd://2.2.1
k8s-worker-2   Ready    <none>          5d21h   v1.37.2   172.31.1.91    <none>        Ubuntu 24.04.5 LTS   7.0.0-1014-aws (amd64)   containerd://2.2.1
ubuntu@k8s-master:~$ kubectl create namespace dev-environment
namespace/dev-environment created
ubuntu@k8s-master:~$ kubectl get namespaces
NAME              STATUS   AGE
default           Active   5d21h
dev-environment   Active   14s
kube-flannel      Active   5d21h
kube-node-lease   Active   5d21h
kube-public       Active   5d21h
kube-system       Active   5d21h
ubuntu@k8s-master:~$ nano dev-quota.yaml
ubuntu@k8s-master:~$ kubectl apply -f dev-quota.yaml
resourcequota/dev-resource-quota created
ubuntu@k8s-master:~$ kubectl describe resourcequota dev-resource-quota -n dev-environment
Name:       dev-resource-quota
Namespace:  dev-environment
Resource    Used  Hard
--------    ----  ----
```



```
ubuntu@k8s-master:~$ kubectl apply -f dev-quota.yaml
resourcequota/dev-resource-quota created
ubuntu@k8s-master:~$ kubectl describe resourcequota dev-resource-quota -n dev-environment
Name:       dev-resource-quota
Namespace:  dev-environment
Resource    Used  Hard
--------    ----  ----
pods        0     3
services    0     2
ubuntu@k8s-master:~$ kubectl run test-pod-1 --image=nginx -n dev-environment
kubectl run test-pod-2 --image=nginx -n dev-environment
kubectl run test-pod-3 --image=nginx -n dev-environment
pod/test-pod-1 created
pod/test-pod-2 created
pod/test-pod-3 created
ubuntu@k8s-master:~$ kubectl run test-pod-4 --image=nginx -n dev-environment
Error from server (Forbidden): pods "test-pod-4" is forbidden: exceeded quota: dev-resource-quota, requested: pods=1, used: pods=3, limited: pods=3
ubuntu@k8s-master:~$ kubectl get pods -n dev-environment
kubectl describe resourcequota dev-resource-quota -n dev-environment
NAME         READY   STATUS    RESTARTS   AGE
test-pod-1   1/1     Running   0          70s
test-pod-2   1/1     Running   0          70s
test-pod-3   1/1     Running   0          70s
Name:       dev-resource-quota
Namespace:  dev-environment
Resource    Used  Hard
--------    ----  ----
pods        3     3
services    0     2
ubuntu@k8s-master:~$ kubectl create service clusterip test-service-1 --tcp=80:80 -n dev-environment
```



```
services    0     2
ubuntu@k8s-master:~$ kubectl create service clusterip test-service-1 --tcp=80:80 -n dev-environment
service/test-service-1 created
ubuntu@k8s-master:~$ kubectl create service clusterip test-service-2 --tcp=80:80 -n dev-environment
service/test-service-2 created
ubuntu@k8s-master:~$ kubectl create service clusterip test-service-3 --tcp=80:80 -n dev-environment
error: failed to create ClusterIP service: services "test-service-3" is forbidden: exceeded quota: dev-resource-quota, requested: services=1, used: services=2, limited:
services=2
ubuntu@k8s-master:~$ kubectl get services -n dev-environment
kubectl describe resourcequota dev-resource-quota -n dev-environment
NAME           TYPE        CLUSTER-IP       EXTERNAL-IP   PORT(S)   AGE
test-service-1   ClusterIP   10.104.82.85     <none>        80/TCP    68s
test-service-2   ClusterIP   10.100.126.146   <none>        80/TCP    55s
Name:       dev-resource-quota
Namespace:  dev-environment
Resource    Used  Hard
--------    ----  ----
pods        3     3
services    2     2
ubuntu@k8s-master:~$ kubectl delete pods --all -n dev-environment
kubectl delete services --all -n dev-environment
pod "test-pod-1" deleted from dev-environment namespace
pod "test-pod-2" deleted from dev-environment namespace
pod "test-pod-3" deleted from dev-environment namespace
service "test-service-1" deleted from dev-environment namespace
service "test-service-2" deleted from dev-environment namespace
ubuntu@k8s-master:~$ kubectl get all -n dev-environment
kubectl describe resourcequota dev-resource-quota -n dev-environment
No resources found in dev-environment namespace.
Name:       dev-resource-quota
```

```
ubuntu@k8s-master:~$ kubectl delete pods --all -n dev-environment
kubectl delete services --all -n dev-environment
pod "test-pod-1" deleted from dev-environment namespace
pod "test-pod-2" deleted from dev-environment namespace
pod "test-pod-3" deleted from dev-environment namespace
service "test-service-1" deleted from dev-environment namespace
service "test-service-2" deleted from dev-environment namespace
ubuntu@k8s-master:~$ kubectl get all -n dev-environment
kubectl describe resourcequota dev-resource-quota -n dev-environment
No resources found in dev-environment namespace.
Name:       dev-resource-quota
Namespace:  dev-environment
Resource    Used  Hard
--------    ----  ----
pods        0     3
services    0     2
ubuntu@k8s-master:~$
```

2. **Create a pod in the prod-environment namespace with 0.2 CPU and 200Mi memory requests, and 0.5 CPU and 500Mi memory limits.**

### K8S-05 – Task 2: Create Pod with CPU and Memory Requests and Limits

## 1. Objective

To create a Kubernetes Pod in the `prod-environment` namespace and configure CPU and memory resource requests and limits.

Requirements:

| Resource | Request | Limit |
|---|---|---|
| CPU | 0.2 CPU (200m) | 0.5 CPU (500m) |
| Memory | 200Mi | 500Mi |

## 2. Step-by-step implementation

### Step 1: Create namespace

Command:

```
None
kubectl create namespace prod-environment
```

Output:

```
None
namespace/prod-environment created
```

### Step 2: Verify namespace

Command:

```
None
kubectl get namespaces
```

Output confirmed:

```
None
prod-environment   Active
```

### Step 3: Create Pod YAML file

Create the YAML file:

```
None
nano prod-resource-pod.yaml
```

Add the following configuration:

```
None
      apiVersion: v1
      kind: Pod
      metadata:
        name: prod-resource-pod
        namespace: prod-environment
      spec:
        containers:
          - name: nginx
            image: nginx
            resources:
              requests:
                cpu: "200m"
                memory: "200Mi"
              limits:
                cpu: "500m"
                memory: "500Mi"
```

Save the file using `Ctrl+O`, Enter, and `Ctrl+X`.

### Step 4: Understand YAML configuration

| YAML field | Description |
| --- | --- |
| `apiVersion: v1` | Uses Kubernetes core API |
| `kind: Pod` | Creates a Pod |
| `metadata.name` | Pod name |

| | |
|---|---|
| `metadata.namespace` | Namespace where the Pod is created |
| `containers` | Defines containers inside the Pod |
| `image: nginx` | Uses the Nginx container image |
| `resources.requests` | Specifies the resources required for scheduling |
| `resources.limits` | Specifies maximum resource usage |

### Step 5: Apply Pod configuration

Command:

```
None
kubectl apply -f prod-resource-pod.yaml
```

Output:

```
None
pod/prod-resource-pod created
```

The Pod was created successfully.

### Step 6: Verify Pod status

Command:

```
None
kubectl get pods -n prod-environment -o wide
```

Output:

```
None
NAME                READY   STATUS    RESTARTS   AGE
prod-resource-pod   1/1     Running   0          14s
```

Pod details:

| Property | Value |
|---|---|
| Pod Name | prod-resource-pod |
| Namespace | prod-environment |
| Status | Running |
| Pod IP | 10.244.1.55 |
| Worker Node | k8s-worker-1 |
| Container | nginx |

### Step 7: Verify CPU and memory settings

Command:

```
None
      kubectl describe pod prod-resource-pod -n prod-environment
```

Important output:

```
None
      Containers:
        nginx:
          Image: nginx

          Limits:
            cpu:     500m
            memory:  500Mi

          Requests:
            cpu:     200m
            memory:  200Mi
```

This confirms that Kubernetes applied the requested CPU and memory configurations correctly.

## 3. Understanding CPU and memory requests and limits

### CPU requests

CPU request specifies how much CPU the container needs for scheduling purposes.

Example:

```
None
      requests:
        cpu: "200m"
```

- `200m` means 200 millicores.
- 1000m = 1 CPU core.
- 200m = 0.2 CPU core.

### CPU limits

CPU limit specifies the maximum CPU the container can use.

Example:

```
None
      limits:
        cpu: "500m"
```

This means the container can use up to 0.5 CPU core. If it tries to use more, Kubernetes enforces the limit through CPU throttling.

### Memory requests

Memory request specifies the amount of memory Kubernetes considers when scheduling the Pod.

Example:

```
None
      requests:
        memory: "200Mi"
```

The scheduler uses this value when checking whether a node has sufficient allocatable memory.

### Memory limits

Memory limit specifies the maximum memory the container can use.

Example:

```
None
      limits:
        memory: "500Mi"
```

If the container exceeds its memory limit, it may be terminated with an `OOMKilled` status.

## 4. Understanding QoS Class

The `kubectl describe` output showed:

```
None
      QoS Class: Burstable
```

Kubernetes assigned the Pod the Burstable QoS class because CPU and memory requests and limits are configured, but their values are different.

| QoS class | Explanation |
| --- | --- |
| Guaranteed | Every container has equal CPU and memory requests and limits |
| Burstable | Requests and limits are configured, but they are not equal or do not meet Guaranteed conditions |
| BestEffort | No CPU or memory requests and limits are configured |

Our Pod belongs to the Burstable category.

## 5. Final result

Task successfully completed

| Requirement | Expected value | Actual result |
| --- | --- | --- |
| Namespace | prod-environment | Created |

| | | |
|---|---|---|
| Pod | prod-resource-pod | Created |
| Container image | nginx | nginx |
| CPU request | 200m | 200m |
| CPU limit | 500m | 500m |
| Memory request | 200Mi | 200Mi |
| Memory limit | 500Mi | 500Mi |
| Pod status | Running | Running |
| QoS class | Burstable | Burstable |

## 6. Important concepts learned

1. CPU and memory requests help Kubernetes schedule Pods.
2. CPU and memory limits restrict container resource consumption.
3. CPU is measured in cores or millicores (`m`).
4. Memory is measured in units such as Mi and Gi.
5. CPU exceeding its limit generally causes throttling.
6. Memory exceeding its limit can cause an OOM kill.
7. Kubernetes uses QoS classes to classify Pods based on their resource configuration.

## 7. Interview questions

Q1. What is the difference between requests and limits?

Requests specify the resources needed for scheduling. Limits specify the maximum resources the container can consume.

Q2. What does 200m CPU mean?

It means 0.2 CPU core or 200 millicores.

Q3. What happens when a container exceeds its CPU limit?

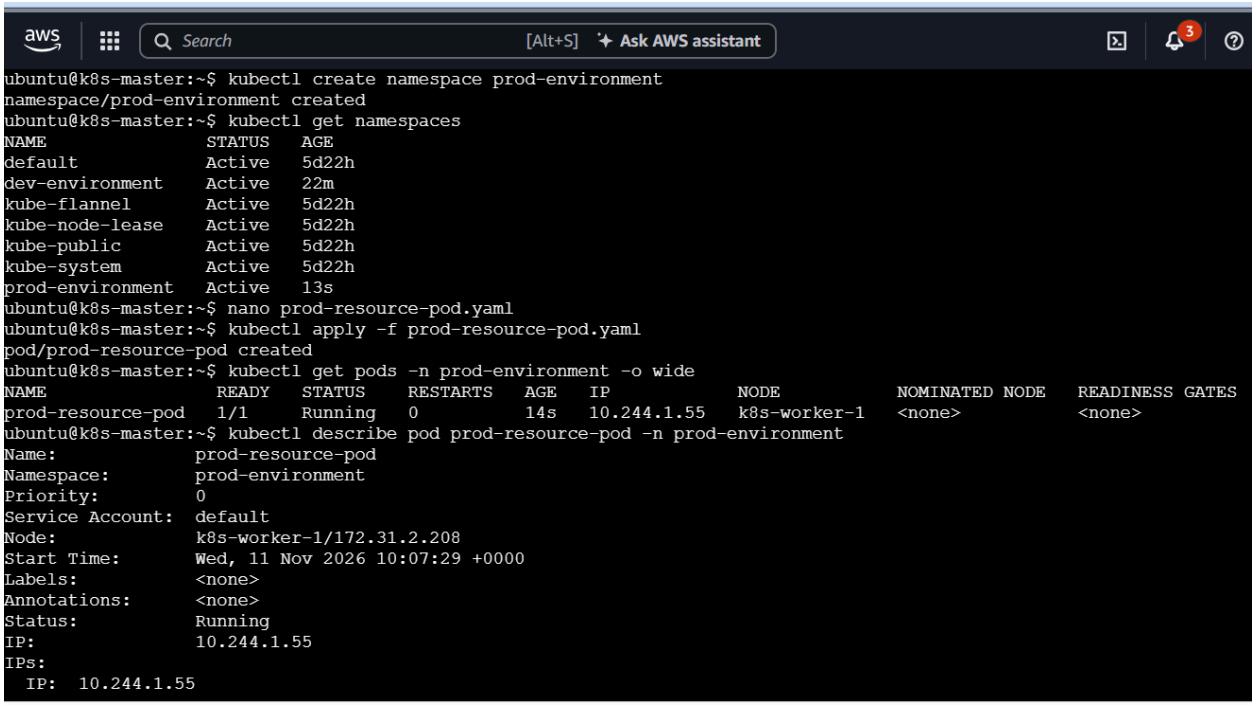
Kubernetes throttles CPU usage.

Q4. What happens when a container exceeds its memory limit?

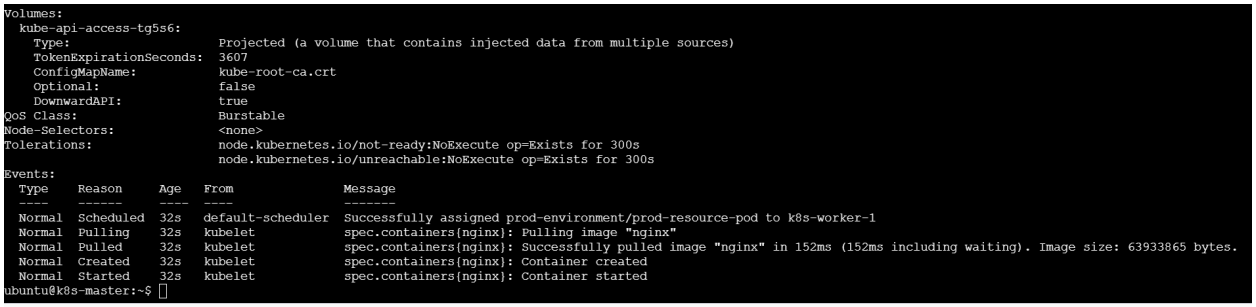
The container may be terminated due to an out-of-memory condition (OOMKilled).

Q5. What is Burstable QoS?

It is a QoS class assigned when resource requests and limits are configured but do not satisfy the Guaranteed QoS conditions.






Task 2 documentation is complete

3. In the staging-environment namespace, set a LimitRange that assigns default CPU and memory limits (300m CPU, 600Mi memory) and applies a minimum and maximum CPU.

# K8S-05 – Task 3 Documentation

## Configure LimitRange with default CPU and memory limits in Kubernetes

Completed successfully

# 1. Objective

To create a Kubernetes LimitRange in the `staging-environment` namespace to:

1. Set default CPU and memory limits for containers.
2. Set default CPU and memory requests.
3. Define minimum and maximum CPU values.
4. Verify that Kubernetes automatically assigns default resource values when a Pod is created without resource specifications.

# 2. What is LimitRange?

A LimitRange is a Kubernetes object used to control the resource requests and limits of individual containers or Pods within a namespace.

It helps administrators define resource boundaries and default values.

For example, suppose multiple developers create Pods in a staging namespace. One developer might create a container that consumes too much CPU or memory.

Using LimitRange, the administrator can define default, minimum and maximum resource values.

Simple example: Think of LimitRange as a rulebook for resource usage in a namespace.

# 3. Step-by-step implementation

## Step 1: Create the namespace

Command:

```
None
kubectl create namespace staging-environment
```

Output:

```
None
namespace/staging-environment created
```

Verify the namespace:

```
None
      kubectl get namespaces
```

The output confirmed that `staging-environment` was in Active status.

Explanation: A separate namespace was created to configure and test resource limits for staging workloads.

### Step 2: Create the LimitRange YAML file

Command:

```
None
      nano staging-limitrange.yaml
```

Configuration:

```
None
      apiVersion: v1
      kind: LimitRange
      metadata:
        name: staging-resource-limits
        namespace: staging-environment
      spec:
        limits:
          - type: Container
            default:
              cpu: "300m"
              memory: "600Mi"
            defaultRequest:
              cpu: "100m"
              memory: "200Mi"
            min:
```

```
      cpu: "100m"
    max:
      cpu: "500m"
```

Save the file.

Explanation of YAML fields:

| Field | Meaning |
| --- | --- |
| `apiVersion: v1` | Kubernetes API version |
| `kind: LimitRange` | Defines the resource as a LimitRange |
| `metadata.name` | Name of the LimitRange |
| `metadata.namespace` | Namespace where it applies |
| `type: Container` | Rules apply to individual containers |
| `default` | Default resource limits |
| `defaultRequest` | Default resource requests |

| | |
|---|---|
| `min` | Minimum permitted resource value |
| `max` | Maximum permitted resource value |

### Step 3: Apply the LimitRange

Command:

```
None
kubectl apply -f staging-limitrange.yaml
```

Output:

```
None
limitrange/staging-resource-limits created
```

Explanation: Kubernetes created the LimitRange configuration in the `staging-environment` namespace.

### Step 4: Verify the LimitRange

Command:

```
None
kubectl describe limitrange staging-resource-limits -n
staging-environment
```

Output:

```
None
Name:       staging-resource-limits
Namespace:  staging-environment
```

```
Type        Resource  Min   Max   Default Request  Default
Limit
----        --------  ---   ---   ---------------
-------------
Container   cpu       100m  500m  100m             300m
Container   memory    -     -     200Mi            600Mi
```

Explanation:

| Resource | Minimum | Maximum | Default request | Default limit |
|---|---|---|---|---|
| CPU | 100m | 500m | 100m | 300m |
| Memory | Not configured | Not configured | 200Mi | 600Mi |

Note: The assignment did not specify exact minimum and maximum CPU values, so we selected 100m and 500m.

### Step 5: Create a Pod without resource specifications

Command:

```
None
kubectl run staging-test-pod --image=nginx -n
staging-environment
```

Output:

```
None
pod/staging-test-pod created
```

Explanation: We created an Nginx Pod without specifying CPU or memory requests and limits.

Normally, if no defaults are configured, the container would not automatically receive these particular resource values.

However, because we configured LimitRange, Kubernetes automatically assigned the default values.

### Step 6: Verify the Pod status

Command:

```
None
kubectl get pods -n staging-environment
```

Output:

```
None
NAME               READY   STATUS    RESTARTS
staging-test-pod   1/1     Running   0
```

Explanation:

- `1/1`: Container is ready.
- `Running`: Pod is running successfully.
- `0`: No container restarts.

### Step 7: Verify automatically assigned resources

Command:

```
None
kubectl describe pod staging-test-pod -n staging-environment
```

Important output:

```
None
Limits:
  cpu:     300m
  memory:  600Mi
```

```
    Requests:
      cpu:     100m
      memory:  200Mi
```

Another important output:

```
None
    Annotations:
      kubernetes.io/limit-ranger:
        LimitRanger plugin set: cpu, memory request
        for container staging-test-pod;
        cpu, memory limit for container staging-test-pod
```

Explanation:

This confirms that Kubernetes automatically assigned the configured default resource requests and limits.

The `LimitRanger` admission plugin applied these values when the Pod was created.

## 4. Understanding CPU and memory values

| Value | Meaning |
| --- | --- |
| `100m` CPU | 0.1 CPU core |
| `300m` CPU | 0.3 CPU core |
| `500m` CPU | 0.5 CPU core |

| | |
|---|---|
| `200Mi` memory | 200 mebibytes |
| `600Mi` memory | 600 mebibytes |

Remember:

- CPU is measured in cores or millicores (`m`).
- Memory is measured in units such as Mi and Gi.

## 5. How LimitRange works

Pod creation request

Developer creates an Nginx Pod without resource settings

LimitRange admission plugin

Checks namespace resource rules and applies defaults

Pod created with default values

CPU request: 100m | CPU limit: 300m

Memory request: 200Mi | Memory limit: 600Mi

Pod Running

## 6. Important concepts learned

| Concept | Explanation |
|---|---|
| LimitRange | Controls individual container or Pod resource settings within a namespace |

| | |
|---|---|
| Default | Automatically assigned resource limit |
| DefaultRequest | Automatically assigned resource request |
| Min | Minimum permitted resource value |
| Max | Maximum permitted resource value |
| LimitRanger | Admission plugin that applies and validates LimitRange rules |
| Namespace | Scope in which the LimitRange applies |

Important: A request is used by the Kubernetes scheduler when selecting a suitable node. A limit specifies the maximum resource usage allowed for the container.

## 7. Difference between ResourceQuota and LimitRange

| ResourceQuota | LimitRange |
|---|---|
| Controls total resource consumption in a namespace | Controls individual container or Pod resource values |
| Can limit total number of Pods and Services | Can set minimum and maximum CPU/memory |
| Controls aggregate resource usage | Can assign default requests and limits |

Example: Maximum 3 Pods

Example: Default CPU limit 300m

## 8. Final verification

Task 3 successful

| Activity | Result |
| --- | --- |
| Created `staging-environment` namespace | Success |
| Created `staging-limitrange.yaml` | Success |
| Applied LimitRange | Success |
| Verified minimum and maximum CPU | Success |
| Created Nginx test Pod | Success |
| Verified automatic CPU defaults | Success |
| Verified automatic memory defaults | Success |
| Pod status | Running |

## 9. Interview questions and answers

Q1. What is LimitRange in Kubernetes?

LimitRange is used to define default, minimum and maximum resource requests and limits for containers or Pods in a namespace.

Q2. What is the difference between requests and limits?

Requests are used by the scheduler to select a suitable node. Limits define the maximum resource usage allowed for a container.

Q3. What happens when we create a Pod without CPU and memory values?

If a LimitRange has default values configured, Kubernetes automatically assigns those values to the container.

Q4. Can we configure LimitRange for a namespace?

Yes. A LimitRange is created inside a namespace and applies to resources in that namespace.

Q5. What is the purpose of the LimitRanger admission plugin?

It applies default resource values and validates resource settings against LimitRange rules during resource creation.

Q6. What is the difference between default and defaultRequest?

- `default`: Automatically assigns resource limits.
- `defaultRequest`: Automatically assigns resource requests.

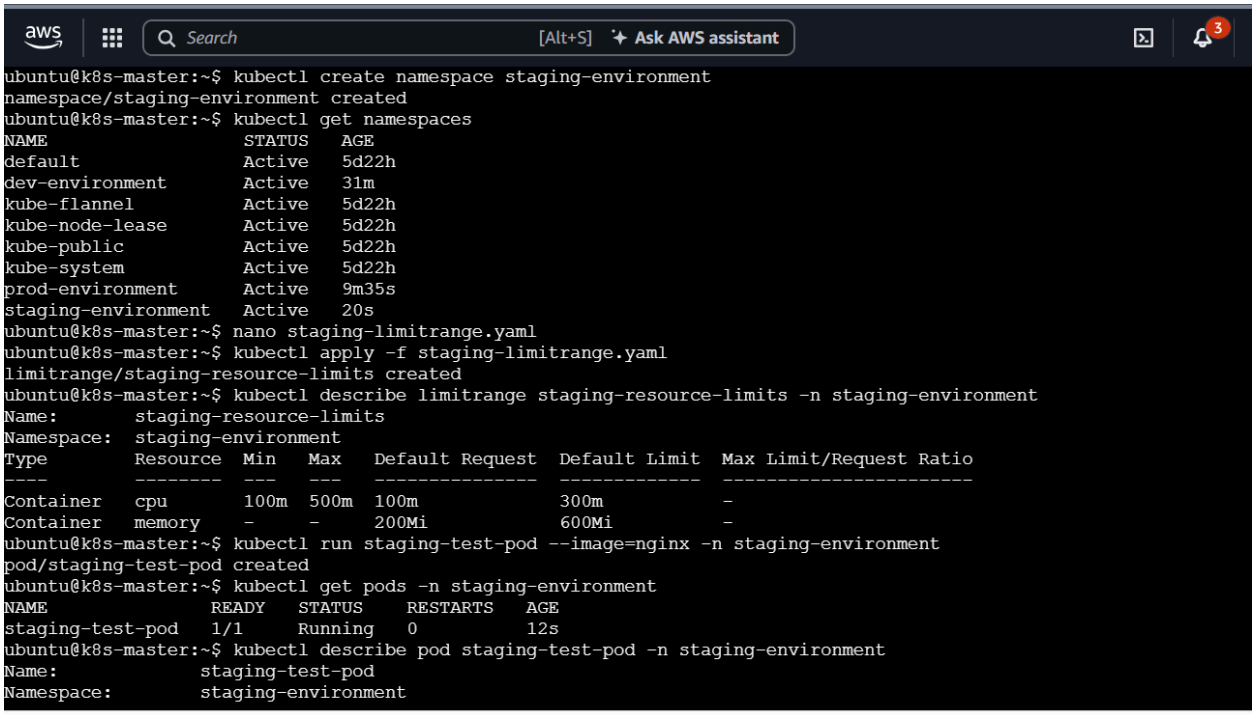
Q7. What happens if a container exceeds the configured maximum CPU value in its resource specification?

Kubernetes rejects the Pod creation or update if the specified value violates the LimitRange maximum.

## 10. Final result

Successfully configured and tested a Kubernetes LimitRange in the `staging-environment` namespace.

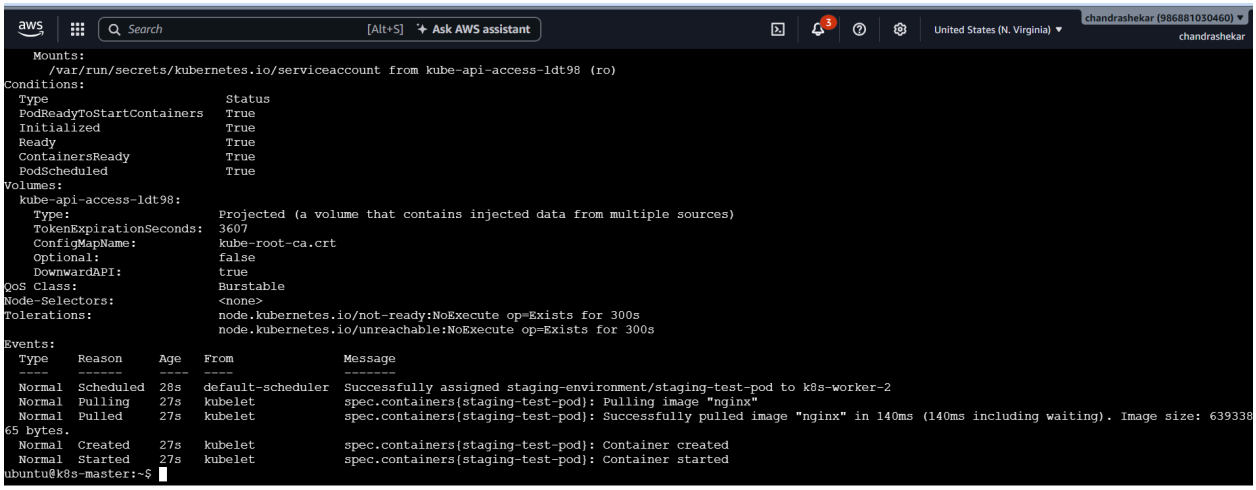
The test confirmed that Kubernetes automatically assigned CPU and memory requests and limits to a Pod even though they were not specified in the Pod creation command.




```
ubuntu@k8s-master:~$ kubectl create namespace staging-environment
namespace/staging-environment created
ubuntu@k8s-master:~$ kubectl get namespaces
NAME                  STATUS   AGE
default               Active   5d22h
dev-environment       Active   31m
kube-flannel          Active   5d22h
kube-node-lease       Active   5d22h
kube-public           Active   5d22h
kube-system           Active   5d22h
prod-environment      Active   9m35s
staging-environment   Active   20s
ubuntu@k8s-master:~$ nano staging-limitrange.yaml
ubuntu@k8s-master:~$ kubectl apply -f staging-limitrange.yaml
limitrange/staging-resource-limits created
ubuntu@k8s-master:~$ kubectl describe limitrange staging-resource-limits -n staging-environment
Name:       staging-resource-limits
Namespace:  staging-environment
Type        Resource  Min   Max   Default Request  Default Limit  Max Limit/Request Ratio
----        --------  ---   ---   ---------------  -------------  -----------------------
Container   cpu       100m  500m  100m             300m           -
Container   memory    -     -     200Mi            600Mi          -
ubuntu@k8s-master:~$ kubectl run staging-test-pod --image=nginx -n staging-environment
pod/staging-test-pod created
ubuntu@k8s-master:~$ kubectl get pods -n staging-environment
NAME               READY   STATUS    RESTARTS   AGE
staging-test-pod   1/1     Running   0          12s
ubuntu@k8s-master:~$ kubectl describe pod staging-test-pod -n staging-environment
Name:             staging-test-pod
Namespace:        staging-environment
```




```
Name:             staging-test-pod
Namespace:        staging-environment
Priority:         0
Service Account:  default
Node:             k8s-worker-2/172.31.1.91
Start Time:       Wed, 11 Nov 2026 10:17:54 +0000
Labels:           run=staging-test-pod
Annotations:      kubernetes.io/limit-ranger:
                    LimitRanger plugin set: cpu, memory request for container staging-test-pod; cpu, memory limit for container staging-test-pod
Status:           Running
IP:               10.244.2.54
IPs:
  IP:  10.244.2.54
Containers:
  staging-test-pod:
    Container ID:   containerd://fb30d0dbd5104291aa983f22037ecad75823ca9df854d615cc89e0be685f828f
    Image:          nginx
    Image ID:       docker.io/library/nginx@sha256:165ddde8751e75a1c907d24ef10d9f6deb08b57e0fe56caf67512486aff76d82
    Port:           <none>
    Host Port:      <none>
    State:          Running
      Started:      Wed, 11 Nov 2026 10:17:55 +0000
    Ready:          True
    Restart Count:  0
    Limits:
      cpu:     300m
      memory:  600Mi
    Requests:
      cpu:        100m
      memory:     200Mi
```




Task 3 is completed

4. **Create a pod and a NodePort service in the default namespace, then create another pod in the test namespace and communicate between them using Service DNS.**

# Cross-Namespace Communication Using Kubernetes Service DNS

## Objective

To create a Pod and NodePort Service in the `default` namespace and access the application from another Pod running in the `test` namespace using Kubernetes Service DNS.

The main objectives are:

1. Create a `test` namespace.
2. Create an Nginx Pod in the `default` namespace.
3. Expose Nginx using a NodePort Service.
4. Create a client Pod in the `test` namespace.
5. Communicate between namespaces using the Service DNS name.
6. Verify successful communication by receiving the Nginx welcome page.

## 2. What is cross-namespace communication?

Kubernetes uses namespaces to organize and separate resources within a cluster.

By default, a Pod can communicate with Services in another namespace, provided network policies or other restrictions do not block the traffic.

For example:

- Nginx application runs in the `default` namespace.
- Client application runs in the `test` namespace.
- The client accesses Nginx using its Service DNS name.

Simple example: Think of namespaces as separate departments in a company. Even though employees belong to different departments, they can communicate using the correct address.

## 3. Step-by-step implementation

### Step 1: Create the test namespace

Command:

```
None
      kubectl create namespace test
```

Output:

```
None
      namespace/test created
```

Verify:

```
None
      kubectl get namespaces
```

Output confirmed:

```
None
      test   Active
```

Explanation: Created a separate namespace named `test` to run the client Pod.

### Step 2: Create an Nginx Pod in the default namespace

Command:

```
None
kubectl run nginx-dns-pod --image=nginx --labels="app=nginx"
```

Initially, the Pod was not found when checking it. We recreated it using a dedicated label to avoid interference from an existing ReplicaSet selector.

Final command:

```
None
kubectl run nginx-dns-pod \
  --image=nginx \
  --labels="app=nginx-dns,tier=backend"
```

Output:

```
None
pod/nginx-dns-pod created
```

Verify:

```
None
kubectl get pod nginx-dns-pod --show-labels
```

Output:

```
None
NAME            READY   STATUS    RESTARTS   AGE   LABELS
nginx-dns-pod   1/1     Running   0          10s
app=nginx-dns,tier=backend
```

Explanation:

- Created an Nginx Pod in the `default` namespace.

- `app=nginx-dns` is the application label used by the Service selector.
- `tier=backend` prevents the existing ReplicaSet with a `NotIn (backend)` selector from adopting this Pod.

### Step 3: Create a NodePort Service YAML file

Command:

```
None
nano nginx-nodeport-dns.yaml
```

Configuration:

```
None
apiVersion: v1
kind: Service
metadata:
  name: nginx-nodeport-dns
  namespace: default
spec:
  type: NodePort
  selector:
    app: nginx-dns
  ports:
    - port: 80
      targetPort: 80
      nodePort: 31080
```

Save the file.

Explanation of YAML fields:

| Field | Meaning |
| --- | --- |

| | |
|---|---|
| `apiVersion: v1` | Kubernetes API version |
| `kind: Service` | Defines a Service resource |
| `metadata.name` | Service name |
| `metadata.namespace` | Namespace where the Service is created |
| `type: NodePort` | Exposes the Service through a port on each node |
| `selector` | Selects Pods using matching labels |
| `port: 80` | Service port |
| `targetPort: 80` | Port inside the Nginx container |
| `nodePort: 31080` | Port exposed on Kubernetes nodes |

Important: The Service selector `app=nginx-dns` matches the label on `nginx-dns-pod`.

### Step 4: Apply the NodePort Service

Command:

```
None
kubectl apply -f nginx-nodeport-dns.yaml
```

Output:

```
None
service/nginx-nodeport-dns created
```

Verify:

```
None
kubectl get service nginx-nodeport-dns
```

Output:

```
None
NAME                 TYPE       CLUSTER-IP       EXTERNAL-IP
PORT(S)
nginx-nodeport-dns   NodePort   10.106.242.119   <none>
80:31080/TCP
```

Explanation:

- Service type: NodePort
- ClusterIP: `10.106.242.119`
- Service port: `80`
- NodePort: `31080`

The Service provides a stable endpoint for accessing the Nginx Pod.

### Step 5: Verify the Service endpoint

Command:

```
None
kubectl get endpoints nginx-nodeport-dns
```

Output:

```
None
NAME                  ENDPOINTS
nginx-nodeport-dns    10.244.2.55:80
```

Explanation:

The endpoint shows the IP address and port of the backend Pod.

- Pod IP: `10.244.2.55`
- Container port: `80`

This confirms that the Service successfully discovered the Nginx Pod using its label selector.

Note: Kubernetes versions 1.33 and later recommend EndpointSlices instead of the older Endpoints API. The deprecation warning does not mean the Service failed.

### Step 6: Create a client Pod in the test namespace

Command:

```
None
kubectl run dns-client \
  --image=busybox:1.36 \
  --restart=Never \
  -n test \
  --command -- sleep 3600
```

Output:

```
None
pod/dns-client created
```

Verify:

```
kubectl get pods -n test
```

Output:

```
NAME         READY   STATUS    RESTARTS
dns-client   1/1     Running   0
```

Explanation:

- Created a BusyBox client Pod in the `test` namespace.
- BusyBox provides useful Linux networking utilities such as `wget` and `nslookup`.
- `sleep 3600` keeps the client container running so we can execute commands inside it.

### Step 7: Test cross-namespace communication using Service DNS

Command:

```
kubectl exec -n test dns-client -- \
  wget -qO-
http://nginx-nodeport-dns.default.svc.cluster.local
```

Output:

```
<!DOCTYPE html>
<html>
<head>
<title>Welcome to nginx!</title>
</head>
<body>
<h1>Welcome to nginx!</h1>
```

```
<p>If you see this page, nginx is successfully installed and working.</p>
</body>
</html>
```

Communication successful

Explanation:

The client Pod successfully accessed the Nginx application using its Service DNS name.

This confirms:

1. Kubernetes DNS resolved the Service name.
2. The Service forwarded the request to the backend Pod.
3. The Nginx container received the request.
4. Nginx returned the welcome page.

## 4. Understanding Kubernetes Service DNS

The full Service DNS name used was:

```
None
nginx-nodeport-dns.default.svc.cluster.local
```

Breakdown:

| Part | Meaning |
| --- | --- |
| `nginx-nodeport-dns` | Service name |
| `default` | Namespace |

| | |
|---|---|
| `svc` | Service domain |
| `cluster.local` | Kubernetes cluster DNS domain |

General DNS format:

```
None
service-name.namespace.svc.cluster.local
```

For example:

```
None
nginx-nodeport-dns.default.svc.cluster.local
```

Because the client is in the `test` namespace and the Service is in the `default` namespace, we used the fully qualified Service DNS name.

## 5. Understand the communication flow

default namespace

Nginx Pod

`nginx-dns-pod`

Label: app=nginx-dns

Service selector matches the Pod label

NodePort Service

`nginx-nodeport-dns`

ClusterIP: 10.106.242.119 | Port: 80

Resolved through Kubernetes DNS

test namespace

BusyBox client Pod

`dns-client`

Nginx welcome page received

## 6. Important difference: Service port and NodePort

| Port | Value | Purpose |
|---|---|---|
| `port` | 80 | Used by clients to access the Service |
| `targetPort` | 80 | Port of the application container |
| `nodePort` | 31080 | Exposes the Service through each node |

Important: For cross-namespace communication inside Kubernetes, we used Service DNS and port 80, not NodePort 31080.

NodePort is useful for accessing the Service through a node's IP address and NodePort number.

## 7. Important concepts learned

| Concept | Explanation |
|---|---|
| Namespace | Organizes and separates Kubernetes resources |
| Pod | Runs application containers |

| | |
|---|---|
| Service | Provides a stable endpoint for Pods |
| NodePort | Exposes a Service through a port on each node |
| ClusterIP | Internal IP address assigned to a Service |
| Selector | Connects a Service to matching Pods |
| Endpoint | Backend Pod IP and port |
| CoreDNS | Resolves Kubernetes DNS names |
| FQDN | Fully Qualified Domain Name |

## 8. Difference between Pod IP and Service DNS

| Pod IP | Service DNS |
|---|---|
| Identifies an individual Pod | Identifies a Service |
| Can change when a Pod is recreated | Provides a stable name |
| Direct Pod communication | Communication through a Service |

| | |
|---|---|
| Example: `10.244.2.55` | `nginx-nodeport-dns.default.svc.cluster.local` |

In real Kubernetes applications, Services are commonly preferred over direct Pod IP addresses because Pod IPs can change.

## 9. Interview questions and answers

Q1. What is cross-namespace communication?

It means communication between Kubernetes resources located in different namespaces.

Q2. Can a Pod in one namespace access a Service in another namespace?

Yes. A Pod can access another namespace's Service using its fully qualified Service DNS name, provided network restrictions do not block communication.

Q3. What is the Kubernetes Service DNS format?

`service-name.namespace.svc.cluster.local`

Q4. What is CoreDNS?

CoreDNS is the DNS service used in Kubernetes to resolve Service and other cluster DNS names.

Q5. What is the difference between ClusterIP and NodePort?

- ClusterIP: Provides internal cluster access.
- NodePort: Exposes a Service through a port on each node.

Q6. Why do we use Service instead of directly using Pod IP?

Pod IP addresses can change when Pods are recreated. A Service provides a stable endpoint.

Q7. Why did we use BusyBox?

BusyBox is a lightweight container image that provides utilities such as `wget` and `nslookup` for network testing.

Q8. What is a Service selector?

A selector identifies Pods using labels and connects them to a Service.

## 10. Final verification

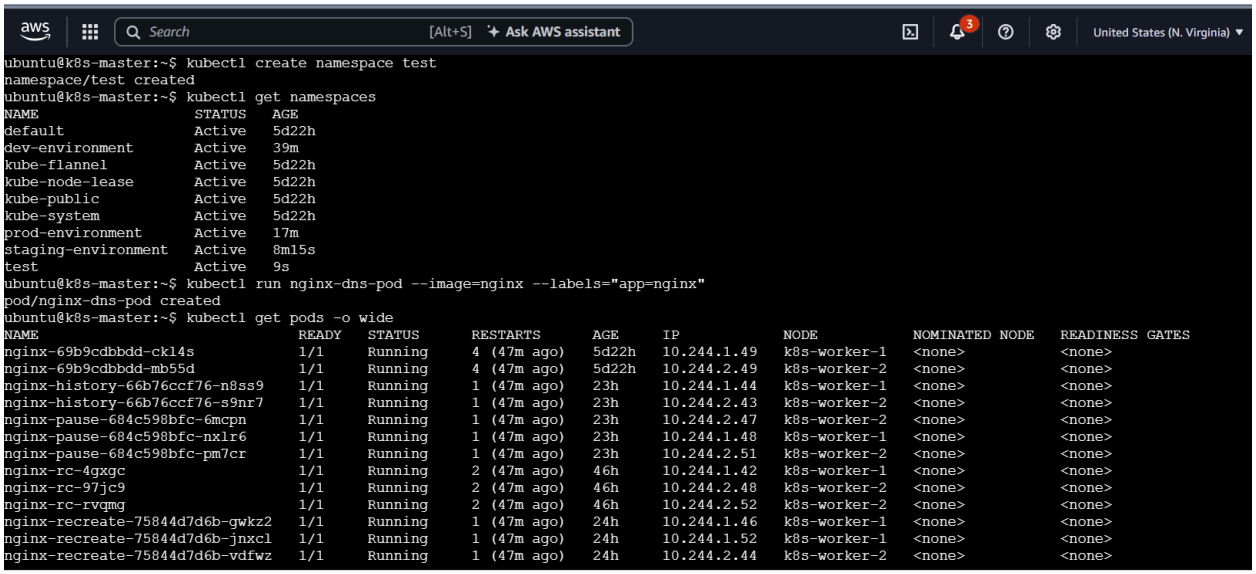
Task 4 completed

| Activity | Result |
| --- | --- |
| Created `test` namespace | Success |
| Created `nginx-dns-pod` in default | Success |
| Configured Pod labels | Success |
| Created NodePort Service | Success |
| Verified Service endpoint | Success |
| Created `dns-client` in test | Success |
| Tested cross-namespace DNS | Success |
| Received Nginx welcome page | Success |

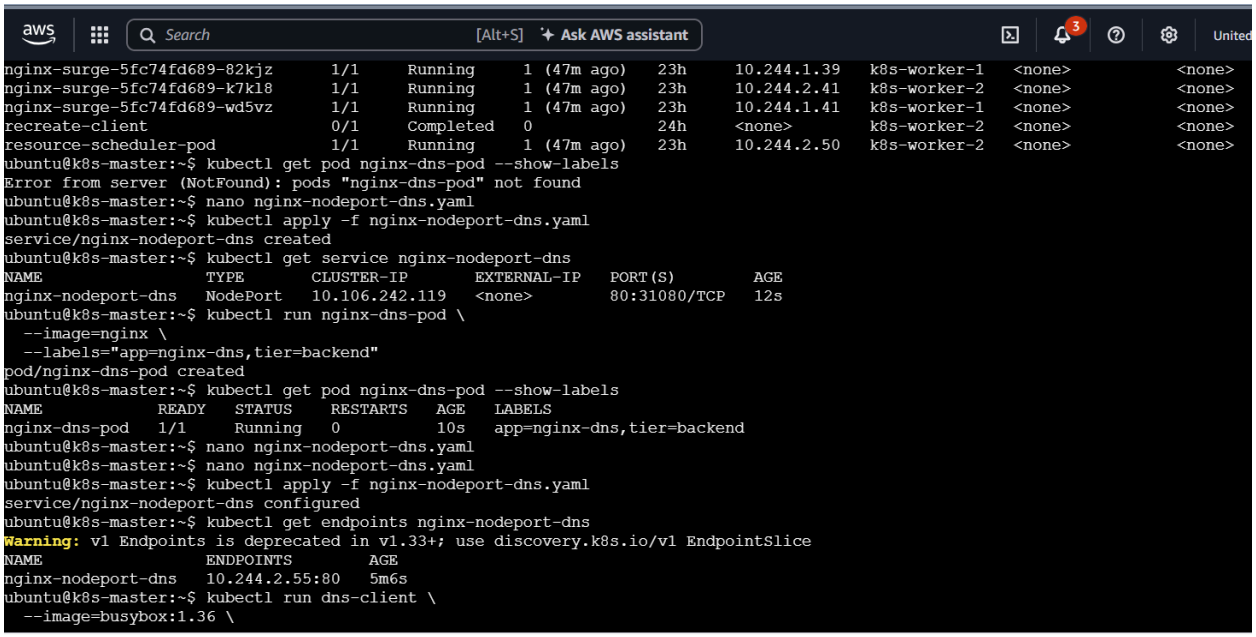
## 11. Final result

Successfully created an Nginx Pod and NodePort Service in the `default` namespace and accessed the application from a BusyBox client Pod in the `test` namespace using Kubernetes Service DNS.

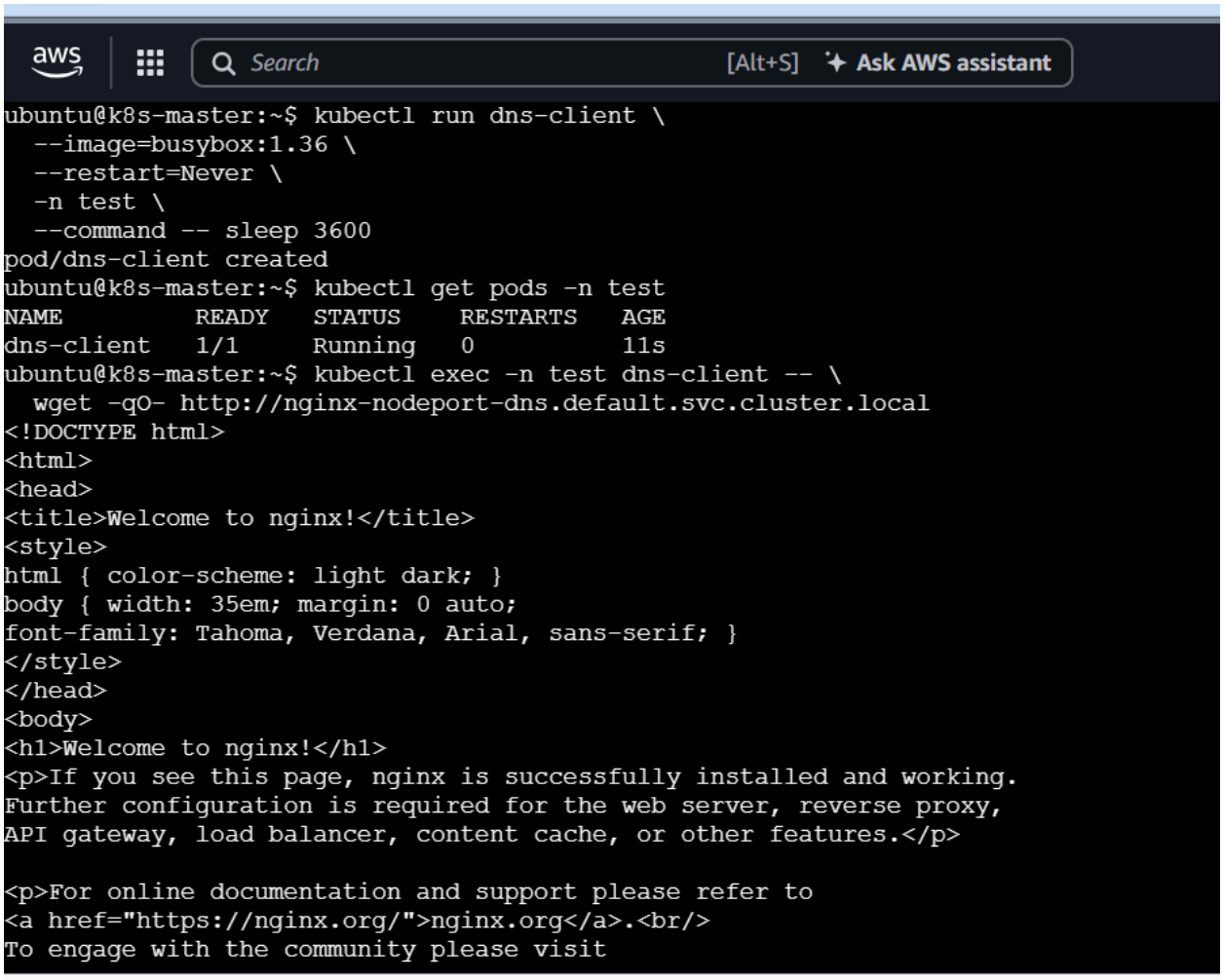
This practical demonstrated how Kubernetes Services, labels, selectors, namespaces and CoreDNS work together to enable communication between applications across namespaces.



```
ubuntu@k8s-master:~$ kubectl create namespace test
namespace/test created
ubuntu@k8s-master:~$ kubectl get namespaces
NAME                 STATUS   AGE
default              Active   5d22h
dev-environment      Active   39m
kube-flannel         Active   5d22h
kube-node-lease      Active   5d22h
kube-public          Active   5d22h
kube-system          Active   5d22h
prod-environment     Active   17m
staging-environment  Active   8m15s
test                 Active   9s
ubuntu@k8s-master:~$ kubectl run nginx-dns-pod --image=nginx --labels="app=nginx"
pod/nginx-dns-pod created
ubuntu@k8s-master:~$ kubectl get pods -o wide
NAME                                 READY   STATUS      RESTARTS      AGE     IP            NODE           NOMINATED NODE   READINESS GATES
nginx-69b9cdbbdd-ckl4s               1/1     Running     4 (47m ago)   5d22h   10.244.1.49   k8s-worker-1   <none>           <none>
nginx-69b9cdbbdd-mb55d               1/1     Running     4 (47m ago)   5d22h   10.244.2.49   k8s-worker-2   <none>           <none>
nginx-history-66b76ccf76-n8ss9       1/1     Running     1 (47m ago)   23h     10.244.1.44   k8s-worker-1   <none>           <none>
nginx-history-66b76ccf76-s9nr7       1/1     Running     1 (47m ago)   23h     10.244.2.43   k8s-worker-2   <none>           <none>
nginx-pause-684c598bfc-6mcpn         1/1     Running     1 (47m ago)   23h     10.244.2.47   k8s-worker-2   <none>           <none>
nginx-pause-684c598bfc-nxlr6         1/1     Running     1 (47m ago)   23h     10.244.1.48   k8s-worker-1   <none>           <none>
nginx-pause-684c598bfc-pm7cr         1/1     Running     1 (47m ago)   23h     10.244.2.51   k8s-worker-2   <none>           <none>
nginx-rc-4gxgc                       1/1     Running     2 (47m ago)   46h     10.244.1.42   k8s-worker-1   <none>           <none>
nginx-rc-97jc9                       1/1     Running     2 (47m ago)   46h     10.244.2.48   k8s-worker-2   <none>           <none>
nginx-rc-rvqmg                       1/1     Running     2 (47m ago)   46h     10.244.2.52   k8s-worker-2   <none>           <none>
nginx-recreate-75844d7d6b-gwkz2      1/1     Running     1 (47m ago)   24h     10.244.1.46   k8s-worker-1   <none>           <none>
nginx-recreate-75844d7d6b-jnxcl      1/1     Running     1 (47m ago)   24h     10.244.1.52   k8s-worker-1   <none>           <none>
nginx-recreate-75844d7d6b-vdfwz      1/1     Running     1 (47m ago)   24h     10.244.2.44   k8s-worker-2   <none>           <none>
```



```
nginx-surge-5fc74fd689-82kjz         1/1     Running     1 (47m ago)   23h     10.244.1.39   k8s-worker-1   <none>           <none>
nginx-surge-5fc74fd689-k7kl8         1/1     Running     1 (47m ago)   23h     10.244.2.41   k8s-worker-2   <none>           <none>
nginx-surge-5fc74fd689-wd5vz         1/1     Running     1 (47m ago)   23h     10.244.1.41   k8s-worker-1   <none>           <none>
recreate-client                      0/1     Completed   0             24h     <none>        k8s-worker-2   <none>           <none>
resource-scheduler-pod               1/1     Running     1 (47m ago)   23h     10.244.2.50   k8s-worker-2   <none>           <none>
ubuntu@k8s-master:~$ kubectl get pod nginx-dns-pod --show-labels
Error from server (NotFound): pods "nginx-dns-pod" not found
ubuntu@k8s-master:~$ nano nginx-nodeport-dns.yaml
ubuntu@k8s-master:~$ kubectl apply -f nginx-nodeport-dns.yaml
service/nginx-nodeport-dns created
ubuntu@k8s-master:~$ kubectl get service nginx-nodeport-dns
NAME                 TYPE       CLUSTER-IP       EXTERNAL-IP   PORT(S)        AGE
nginx-nodeport-dns   NodePort   10.106.242.119   <none>        80:31080/TCP   12s
ubuntu@k8s-master:~$ kubectl run nginx-dns-pod \
  --image=nginx \
  --labels="app=nginx-dns,tier=backend"
pod/nginx-dns-pod created
ubuntu@k8s-master:~$ kubectl get pod nginx-dns-pod --show-labels
NAME            READY   STATUS    RESTARTS   AGE   LABELS
nginx-dns-pod   1/1     Running   0          10s   app=nginx-dns,tier=backend
ubuntu@k8s-master:~$ nano nginx-nodeport-dns.yaml
ubuntu@k8s-master:~$ nano nginx-nodeport-dns.yaml
ubuntu@k8s-master:~$ kubectl apply -f nginx-nodeport-dns.yaml
service/nginx-nodeport-dns configured
ubuntu@k8s-master:~$ kubectl get endpoints nginx-nodeport-dns
Warning: v1 Endpoints is deprecated in v1.33+; use discovery.k8s.io/v1 EndpointSlice
NAME                 ENDPOINTS         AGE
nginx-nodeport-dns   10.244.2.55:80    5m6s
ubuntu@k8s-master:~$ kubectl run dns-client \
  --image=busybox:1.36 \
```


aws
Search
[Alt+S]
Ask AWS assistant
ubuntu@k8s-master:~$ kubectl run dns-client \
--image=busybox:1.36 \
--restart=Never \
-n test \
--command -- sleep 3600
pod/dns-client created
ubuntu@k8s-master:~$ kubectl get pods -n test
NAME READY STATUS RESTARTS AGE
dns-client 1/1 Running 0 11s
ubuntu@k8s-master:~$ kubectl exec -n test dns-client -- \
wget -qO- http://nginx-nodeport-dns.default.svc.cluster.local
<!DOCTYPE html>
<html>
<head>
<title>Welcome to nginx!</title>
<style>
html { color-scheme: light dark; }
body { width: 35em; margin: 0 auto;
font-family: Tahoma, Verdana, Arial, sans-serif; }
</style>
</head>
<body>
<h1>Welcome to nginx!</h1>
If you see this page, nginx is successfully installed and working.
Further configuration is required for the web server, reverse proxy,
API gateway, load balancer, content cache, or other features.
For online documentation and support please refer to
<a href="https://nginx.org/">nginx.org</a>.<br/>
To engage with the community please visit

```
ubuntu@k8s-master:~$ kubectl exec -n test dns-client -- \
  wget -qO- http://nginx-nodeport-dns.default.svc.cluster.local
<!DOCTYPE html>
<html>
<head>
<title>Welcome to nginx!</title>
<style>
html { color-scheme: light dark; }
body { width: 35em; margin: 0 auto;
font-family: Tahoma, Verdana, Arial, sans-serif; }
</style>
</head>
<body>
<h1>Welcome to nginx!</h1>
<p>If you see this page, nginx is successfully installed and working.
Further configuration is required for the web server, reverse proxy,
API gateway, load balancer, content cache, or other features.</p>

<p>For online documentation and support please refer to
<a href="https://nginx.org/">nginx.org</a>.<br/>
To engage with the community please visit
<a href="https://community.nginx.org/">community.nginx.org</a>.<br/>
For enterprise grade support, professional services, additional
security features and capabilities please refer to
<a href="https://f5.com/nginx">f5.com/nginx</a>.</p>

<p><em>Thank you for using nginx.</em></p>
</body>
</html>
ubuntu@k8s-master:~$
```

Task 4 is completed successfully.

5. **Apply a LimitRange with a max limit/request ratio of 2 for memory in the performance-environment namespace, and test by creating a pod with mismatched resource requests and limits.**

# Apply LimitRange with maxLimitRequestRatio

## 1. Objective

To configure a Kubernetes LimitRange that restricts the maximum memory limit-to-request ratio to 2 for every container in the `performance-environment` namespace.

We also tested:

- An invalid Pod with a ratio of 5.
- A valid Pod with a ratio of 2.

### 2. Create the namespace

Command:

```
None
kubectl create namespace performance-environment
```

Output:

```
None
namespace/performance-environment created
```

Verify:

```
None
kubectl get namespaces
```

The `performance-environment` namespace appeared with status `Active`.

### 3. Create the LimitRange YAML

File: `performance-limitrange.yaml`

```
None
apiVersion: v1
kind: LimitRange
metadata:
  name: memory-ratio-limit
  namespace: performance-environment
spec:
  limits:
    - type: Container
      maxLimitRequestRatio:
        memory: "2"
```

Apply:

```
None
kubectl apply -f performance-limitrange.yaml
```

Output:

```
None
limitrange/memory-ratio-limit created
```

### 4. Verify the LimitRange

Command:

```
None
kubectl describe limitrange memory-ratio-limit -n
performance-environment
```

Important output:

```
None
Type        Resource  Max Limit/Request Ratio
Container   memory    2
```

Explanation:

- `type: Container` means the rule applies to each container.
- `memory: "2"` means the memory limit cannot exceed two times the memory request.
- This rule is enforced when Kubernetes validates a new Pod.

Formula:

$$Ratio = \frac{MemoryLimit}{MemoryRequest}$$

The ratio must be less than or equal to 2.

### 5. Test an invalid Pod

File: `memory-invalid-pod.yaml`

```
None
apiVersion: v1
kind: Pod
metadata:
  name: memory-invalid-pod
  namespace: performance-environment
spec:
  containers:
    - name: nginx
      image: nginx
      resources:
        requests:
          memory: "100Mi"
        limits:
          memory: "500Mi"
```

Apply:

```
None
kubectl apply -f memory-invalid-pod.yaml
```

Actual output:

```
None
Error from server (Forbidden):
pods "memory-invalid-pod" is forbidden:
memory max limit to request ratio per Container is 2,
but provided ratio is 5.000000
```

Explanation:

Memory request = 100Mi

Memory limit = 500Mi

Calculation:

$$500Mi \div 100Mi = 5$$

The ratio is 5, which exceeds the maximum allowed ratio of 2.

Therefore, Kubernetes rejected the Pod.

Expected rejection – policy working

## 6. Test a valid Pod

File: `memory-valid-pod.yaml`

```
None
apiVersion: v1
kind: Pod
metadata:
  name: memory-valid-pod
  namespace: performance-environment
spec:
  containers:
    - name: nginx
      image: nginx
      resources:
        requests:
          memory: "100Mi"
        limits:
          memory: "200Mi"
```

Apply:

```
None
kubectl apply -f memory-valid-pod.yaml
```

Output:

```
None
pod/memory-valid-pod created
```

Verify:

```
None
kubectl get pods -n performance-environment
```

Actual output:

```
None
NAME                READY   STATUS    RESTARTS
memory-valid-pod    1/1     Running   0
```

Explanation:

Memory request = 100Mi

Memory limit = 200Mi

Calculation:

$$200Mi \div 100Mi = 2$$

The ratio is exactly 2, so Kubernetes accepted the Pod.

## 7. Comparison of test results

| Pod | Request | Limit | Ratio | Result |
|---|---|---|---|---|
| Invalid Pod | 100Mi | 500Mi | 5 | Rejected |
| Valid Pod | 100Mi | 200Mi | 2 | Accepted |

## 8. Concepts learned

What is maxLimitRequestRatio?

It controls how much higher a container's resource limit can be compared with its request.

Why is it useful?

It helps administrators prevent containers from having excessively high limits compared with their requested resources.

Where is this rule applied?

Inside a namespace through a Kubernetes LimitRange object.

Does it apply to CPU also?

Not in our configuration. We configured only the memory ratio.

Does Kubernetes check this rule?

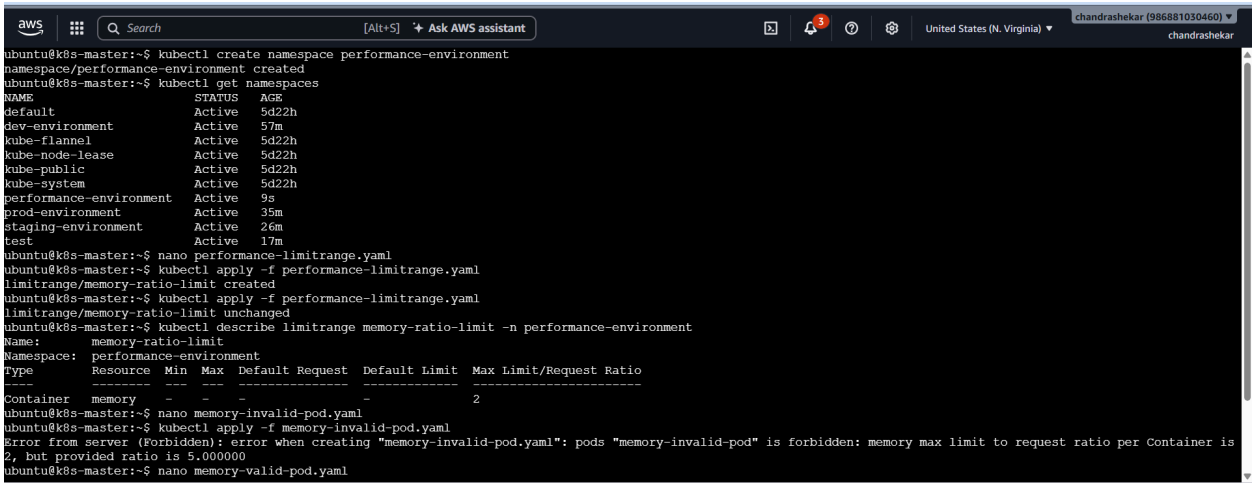
Yes. Kubernetes checks it when a Pod is submitted for creation or update.

# Final result – K8S-05 Task 5

Successfully completed

- Created `performance-environment` namespace.
- Created and applied `memory-ratio-limit`.
- Configured memory `maxLimitRequestRatio: 2`.
- Tested an invalid Pod with ratio 5; Kubernetes rejected it.
- Tested a valid Pod with ratio 2; Kubernetes accepted it.
- Verified that the valid Pod reached `Running` status.

All five K8S-05 practical tasks are now completed.



```
ubuntu@k8s-master:~$ kubectl apply -f memory-invalid-pod.yaml
Error from server (Forbidden): error when creating "memory-invalid-pod.yaml": pods "memory-invalid-pod" is forbidden: memory max limit to request ratio per Container is
2, but provided ratio is 5.000000
ubuntu@k8s-master:~$ nano memory-valid-pod.yaml
ubuntu@k8s-master:~$ kubectl apply -f memory-valid-pod.yaml
pod/memory-valid-pod created
ubuntu@k8s-master:~$ kubectl get pods -n performance-environment
NAME               READY   STATUS    RESTARTS   AGE
memory-valid-pod   1/1     Running   0          11s
ubuntu@k8s-master:~$
```